

Python Documentation: Data Structures

Source: <https://docs.python.org/3/tutorial/datastructures.html>

This chapter describes some things you've learned about already in more detail, and adds some new things as well.

More on Lists

The list data type has some more methods. Here are all of the methods of list objects:

- `list.append(value)` — Add an item to the end of the list. Similar to `a[len(a):] = [x]`.
- `list.extend(iterable)` — Extend the list by appending all the items from the iterable. Similar to `a[len(a):] = iterable`.
- `list.insert(index, value)` — Insert an item at a given position. `a.insert(0, x)` inserts at the front of the list, and `a.insert(len(a), x)` is equivalent to `a.append(x)`.
- `list.remove(value)` — Remove the first item from the list whose value is equal to value. Raises a `ValueError` if there is no such item.
- `list.pop(index=-1)` — Remove the item at the given position in the list, and return it. If no index is specified, `a.pop()` removes and returns the last item in the list. Raises an `IndexError` if the list is empty or the index is outside the list range.
- `list.clear()` — Remove all items from the list. Similar to `del a[:]`.
- `list.index(value, start, stop)` — Return zero-based index of the first occurrence of value in the list. Raises a `ValueError` if there is no such item.
- `list.count(value)` — Return the number of times value appears in the list.
- `list.sort(key=None, reverse=False)` — Sort the items of the list in place.
- `list.reverse()` — Reverse the elements of the list in place.
- `list.copy()` — Return a shallow copy of the list. Similar to `a[:]`.

Example that uses most of the list methods:

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.count('tangerine')
0
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4) # Find next banana starting at position 4
6
>>> fruits.reverse()
>>> fruits.append('grape')
>>> fruits.sort()
>>> fruits.pop()
'pear'
```

Methods like `insert`, `remove` or `sort` that only modify the list have no return value printed — they return the default `None`. This is a design principle for all mutable data structures in Python.

Using Lists as Stacks

The list methods make it very easy to use a list as a stack, where the last element added is the first element retrieved ("last-in, first-out"). To add an item to the top of the stack, use `append()`. To retrieve an item from the top of the stack, use `pop()` without an explicit index.

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
```

Using Lists as Queues

It is also possible to use a list as a queue, where the first element added is the first element retrieved ("first-in, first-out"); however, lists are not efficient for this purpose. While appends and pops from the end of list are fast, doing inserts or pops from the beginning of a list is slow (because all of the other elements have to be shifted by one).

To implement a queue, use `collections.deque` which was designed to have fast appends and pops from both ends.

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # Terry arrives
>>> queue.append("Graham")         # Graham arrives
>>> queue.popleft()                 # The first to arrive now leaves
'Eric'
>>> queue.popleft()                 # The second to arrive now leaves
'John'
>>> queue
deque(['Michael', 'Terry', 'Graham'])
```

List Comprehensions

List comprehensions provide a concise way to create lists. Common applications are to make new lists where each element is the result of some operations applied to each member of another sequence or iterable, or to create a subsequence of those elements that satisfy a certain condition.

```
>>> squares = [x**2 for x in range(10)]
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

A list comprehension consists of brackets containing an expression followed by a `for` clause, then zero or more `for` or `if` clauses.

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]

>>> vec = [-4, -2, 0, 2, 4]
>>> [x*2 for x in vec]
[-8, -4, 0, 4, 8]
>>> [x for x in vec if x >= 0]
[0, 2, 4]
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
```

Nested List Comprehensions

The initial expression in a list comprehension can be any arbitrary expression, including another list comprehension.

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]
>>> [[row[i] for row in matrix] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

The `zip()` function does the same job more idiomatically:

```
>>> list(zip(*matrix))
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

The del statement

There is a way to remove an item from a list given its index instead of its value: the `del` statement. This differs from `pop()` which returns a value.

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> del a[2:4]
>>> del a[:]
>>> a
[]
```

Tuples and Sequences

A tuple consists of a number of values separated by commas:

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
```

Tuples are immutable, and usually contain a heterogeneous sequence of elements that are accessed via unpacking or indexing. Lists are mutable, and their elements are usually homogeneous and are accessed by iterating over the list.

```
>>> x, y, z = t
```

This is called sequence unpacking and works for any sequence on the right-hand side.

Sets

Python also includes a data type for sets. A set is an unordered collection with no duplicate elements. Basic uses include membership testing and eliminating duplicate entries. Set objects also support mathematical operations like union, intersection, difference, and symmetric difference.

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> print(basket)
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket
True
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a - b
# letters in a but not in b
```

```
{'r', 'd', 'b'}
>>> a | b                                # letters in a or b or both
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                                # letters in both a and b
{'a', 'c'}
>>> a ^ b                                # letters in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

Dictionaries

Another useful data type built into Python is the dictionary. Dictionaries are indexed by keys, which can be any immutable type; strings and numbers can always be keys.

It is best to think of a dictionary as a set of key: value pairs, with the requirement that the keys are unique.

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
>>> del tel['sape']
>>> list(tel)
['jack', 'guido']
>>> sorted(tel)
['guido', 'jack']
>>> 'guido' in tel
True
```

The `dict()` constructor builds dictionaries directly from sequences of key-value pairs, and dict comprehensions can be used to create dictionaries from arbitrary key and value expressions:

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
```

Looping Techniques

When looping through dictionaries, the key and corresponding value can be retrieved at the same time using the `items()` method.

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():
...     print(k, v)
gallahad the pure
robin the brave
```

When looping through a sequence, the position index and corresponding value can be retrieved at the same time using the `enumerate()` function.

```
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
0 tic
1 tac
2 toe
```

To loop over two or more sequences at the same time, the entries can be paired with the `zip()` function.

```
>>> questions = ['name', 'quest', 'favorite color']
>>> answers = ['lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
```

```
...     print('What is your {0}? It is {1}'.format(q, a))
```

More on Conditions

The comparison operators `in` and `not in` are membership tests that determine whether a value is in (or not in) a container. The operators `is` and `is not` compare whether two objects are really the same object.

Comparisons can be chained: `a < b == c` tests whether `a` is less than `b` and moreover `b` equals `c`.

The Boolean operators `and` and `or` are short-circuit operators: their arguments are evaluated from left to right, and evaluation stops as soon as the outcome is determined.

Comparing Sequences and Other Types

Sequence objects typically may be compared to other objects with the same sequence type. The comparison uses lexicographical ordering: first the first two items are compared, and if they differ this determines the outcome of the comparison; if they are equal, the next two items are compared, and so on.

```
(1, 2, 3)          < (1, 2, 4)
[1, 2, 3]          < [1, 2, 4]
'ABC' < 'C' < 'Pascal' < 'Python'
(1, 2, 3, 4)       < (1, 2, 4)
(1, 2, 3)          == (1.0, 2.0, 3.0)
```

Source: Python 3 Documentation, © Python Software Foundation. Licensed under the Python Software Foundation License Version 2.